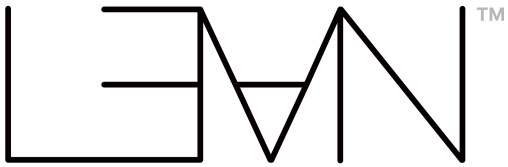
# Lean and formalisation

## Doing mathematics differently

Abhijit A J

University of Paderborn

15th October 2026



# What is Lean?

# What is Lean?

Simply put, Lean is an open-source programming language

# What is Lean?

Simply put, Lean is an open-source programming language and theorem prover built for *formalising maths* and for *formal verification*.

# What is Lean?

Simply put, Lean is an open-source programming language and theorem prover built for *formalising maths* and for *formal verification*.

- Lean is not mainly a tool for discovering proofs (as many LLMs can attempt nowadays). The proof still has to come from the user.

# What is Lean?

Simply put, Lean is an open-source programming language and theorem prover built for *formalising maths* and for *formal verification*.

- Lean is not mainly a tool for discovering proofs (as many LLMs can attempt nowadays). The proof still has to come from the user.
- Although it can compute, it is not a computer algebra system like SageMath or Mathematica.

# What is Lean?

Simply put, Lean is an open-source programming language and theorem prover built for *formalising maths* and for *formal verification*.

- Lean is not mainly a tool for discovering proofs (as many LLMs can attempt nowadays). The proof still has to come from the user.
- Although it can compute, it is not a computer algebra system like SageMath or Mathematica.

It helps you *verify* your definitions and proofs.

# What is formalising maths?

# What is formalising maths?

Formalising mathematics is when you can explain to a computer what different mathematical structures are.

# What is formalising maths?

Formalising mathematics is when you can explain to a computer what different mathematical structures are. For instance, you can say to a computer

*Let $X$ be a topological space...*

and the computer will not be confused, or give you an error.

# What is formalising maths?

Formalising mathematics is when you can explain to a computer what different mathematical structures are. For instance, you can say to a computer

*Let $X$ be a topological space...*

and the computer will not be confused, or give you an error.

One of the biggest achievements (I am biased as someone who works in Number Theory / Arithmetic Geometry) in formalising mathematical definitions was when Kevin Buzzard, Johan Commelin, Patrick Massot and others explained to Lean what a perfectoid space is.

# What is formalising maths?

Formalising mathematics is when you can explain to a computer what different mathematical structures are. For instance, you can say to a computer

*Let $X$ be a topological space...*

and the computer will not be confused, or give you an error.

One of the biggest achievements (I am biased as someone who works in Number Theory / Arithmetic Geometry) in formalising mathematical definitions was when Kevin Buzzard, Johan Commelin, Patrick Massot and others explained to Lean what a perfectoid space is.

What's so hard about this?

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand what an adic space is,

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand what an adic space is, what isomorphic means,

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand what an adic space is, what isomorphic means, what a perfectoid Huber pair is,

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand what an adic space is, what isomorphic means, what a perfectoid Huber pair is, in fact, what a pair is,

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand what an adic space is, what isomorphic means, what a perfectoid Huber pair is, in fact, what a pair is, what a ring is,

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

To understand this definition, one needs to understand what an adic space is, what isomorphic means, what a perfectoid Huber pair is, in fact, what a pair is, what a ring is, what a subring is,

# What is formalising maths?

**Definition**

An **affinoid perfectoid space** is an adic space isomorphic to $\mathrm{Spa}(R, R^+)$ for some perfectoid Huber pair $(R, R^+)$. A **perfectoid space** is an adic space that is locally isomorphic to an affinoid perfectoid space.

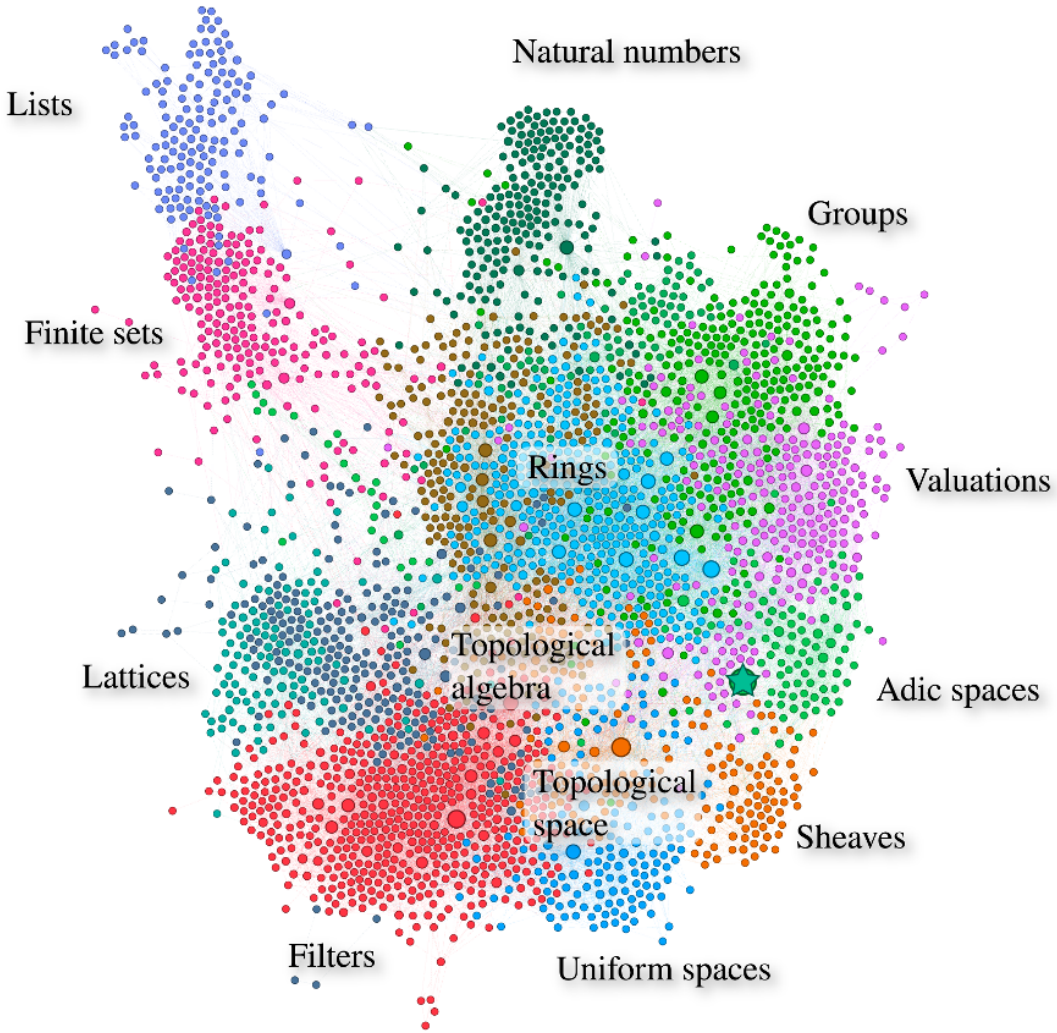
To understand this definition, one needs to understand what an adic space is, what isomorphic means, what a perfectoid Huber pair is, in fact, what a pair is, what a ring is, what a subring is, what a topological space is ...

# What is formalising maths?

There is a graphical representation of all the topics they had to cover, en route to formalising perfectoid spaces:

# What is formalising maths?

There is a graphical representation of all the topics they had to cover, en route to formalising perfectoid spaces:



# What is formal verification?

# What is formal verification?

Formal verification is when one formally checks (using a finite list of commands, axioms and already proven results) all the steps of a proof.

# What is formal verification?

Formal verification is when one formally checks (using a finite list of commands, axioms and already proven results) all the steps of a proof. One of the biggest pre-AI achievements in formal proof verification was the *Liquid Tensor Experiment*.

# What is formal verification?

Formal verification is when one formally checks (using a finite list of commands, axioms and already proven results) all the steps of a proof. One of the biggest pre-AI achievements in formal proof verification was the *Liquid Tensor Experiment*.

In December 2020, Peter Scholze put out a challenge to the theorem prover community to see if they could verify a result of his (and Dustin Clausen's) about liquid vector spaces (amongst other reasons, because he had *'lingering doubts'* about the proof).

# What is formal verification?

Formal verification is when one formally checks (using a finite list of commands, axioms and already proven results) all the steps of a proof. One of the biggest pre-AI achievements in formal proof verification was the *Liquid Tensor Experiment*.

In December 2020, Peter Scholze put out a challenge to the theorem prover community to see if they could verify a result of his (and Dustin Clausen's) about liquid vector spaces (amongst other reasons, because he had *'lingering doubts'* about the proof).

In July 2022, the proof of the theorem was finally verified using Lean by a group of collaborators led by Johan Commelin.

# What is formal verification?

There is currently an ongoing project, led by Kevin Buzzard, to try and verify (a more modern version of) the proof of Fermat's Last Theorem.

# What is formal verification?

There is currently an ongoing project, led by Kevin Buzzard, to try and verify (a more modern version of) the proof of Fermat's Last Theorem.

In September 2026, Anthropic announced that Claude had formalised the original proof of Fermat's Last Theorem, working largely autonomously over 11 days.

# What is formal verification?

There is currently an ongoing project, led by Kevin Buzzard, to try and verify (a more modern version of) the proof of Fermat's Last Theorem.

In September 2026, Anthropic announced that Claude had formalised the original proof of Fermat's Last Theorem, working largely autonomously over 11 days.

In the same month, OpenAI announced a claimed solution (a counterexample) to the Navier-Stokes existence and smoothness problem, together with a Lean formalisation. The claim is being examined by the community, and there is a dispute over credit.

# What is formal verification?

There is currently an ongoing project, led by Kevin Buzzard, to try and verify (a more modern version of) the proof of Fermat's Last Theorem.

In September 2026, Anthropic announced that Claude had formalised the original proof of Fermat's Last Theorem, working largely autonomously over 11 days.

In the same month, OpenAI announced a claimed solution (a counterexample) to the Navier-Stokes existence and smoothness problem, together with a Lean formalisation. The claim is being examined by the community, and there is a dispute over credit.

In late September, mathematicians and AI agents collaborated to produce a Lean formalisation of a proof of the Poincaré conjecture, checked by Lean's kernel and a second, independently built verification tool. Independent mathematical review is still pending.

# The mathematics library `Mathlib`

# The mathematics library `Mathlib`

All the projects mentioned previously follow a top-down approach

# The mathematics library `Mathlib`

All the projects mentioned previously follow a top-down approach, i.e., you have a goal in mind, and you work backwards to try and fill in the gaps to prove the goal.

# The mathematics library `Mathlib`

All the projects mentioned previously follow a top-down approach, i.e., you have a goal in mind, and you work backwards to try and fill in the gaps to prove the goal.

But, probably the most contributed-to project, by members of the Lean theorem prover community, is `Mathlib`, the mathematics library, which follows a bottom-up approach.

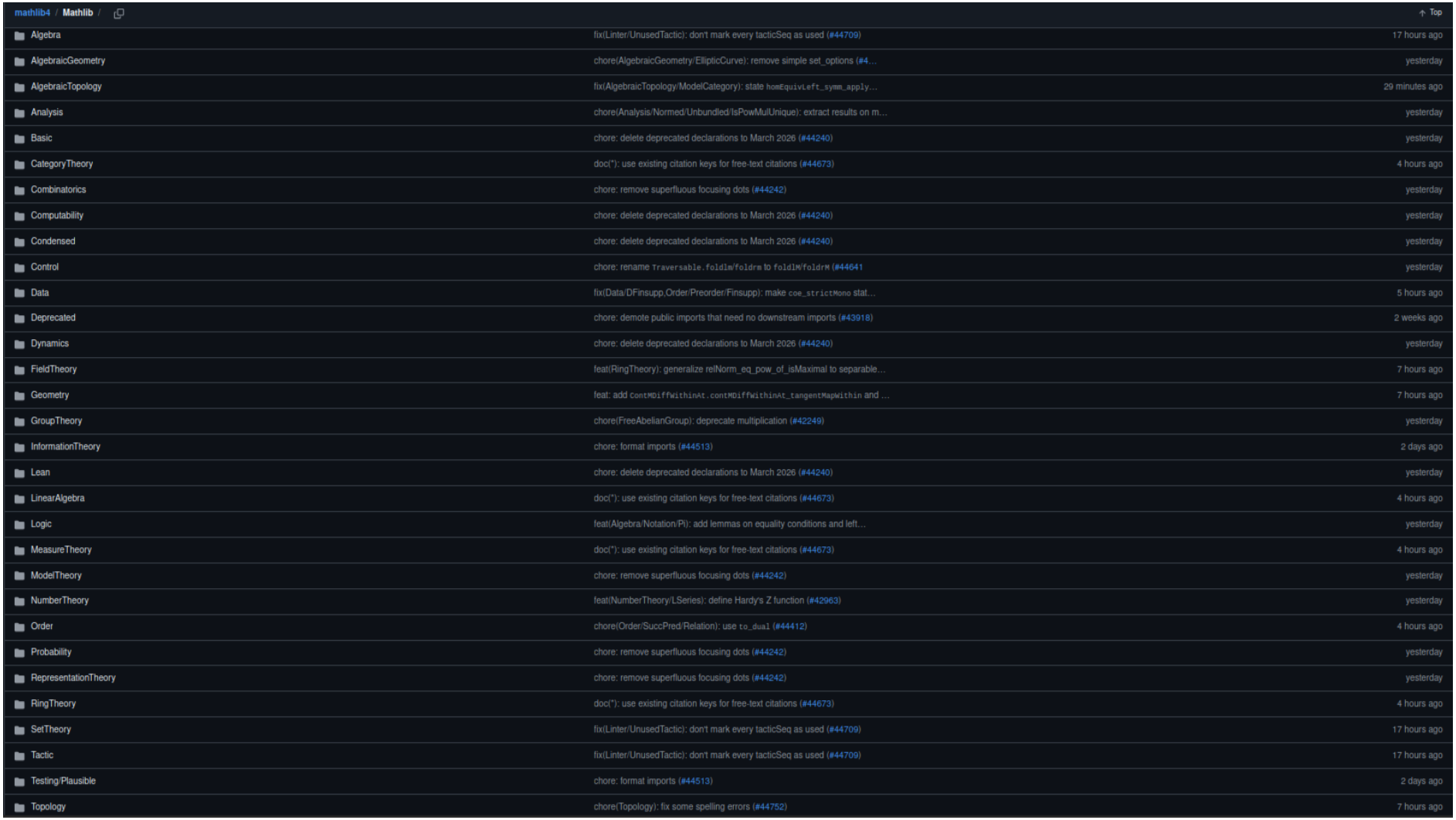
# The mathematics library `Mathlib`

All the projects mentioned previously follow a top-down approach, i.e., you have a goal in mind, and you work backwards to try and fill in the gaps to prove the goal.

But, probably the most contributed-to project, by members of the Lean theorem prover community, is `Mathlib`, the mathematics library, which follows a bottom-up approach.

The library `Mathlib` aims to catalogue 'all' basic structures and results in mathematics that can be used by mathematicians to verify their own, more technical results.

# The mathematics library `Mathlib`



mathlib4 / Mathlib / ↑ Top

| | | |
|---|---|---|
| Algebra | fix(Linter/UnusedTactic): don't mark every tacticSeq as used (#44709) | 17 hours ago |
| AlgebraicGeometry | chore(AlgebraicGeometry/EllipticCurve): remove simple set_options (#4… | yesterday |
| AlgebraicTopology | fix(AlgebraicTopology/ModelCategory): state `homEquivLeft_symm_apply`… | 29 minutes ago |
| Analysis | chore(Analysis/Normed/Unbundled/IsPowMulUnique): extract results on m… | yesterday |
| Basic | chore: delete deprecated declarations to March 2026 (#44240) | yesterday |
| CategoryTheory | doc(*): use existing citation keys for free-text citations (#44673) | 4 hours ago |
| Combinatorics | chore: remove superfluous focusing dots (#44242) | yesterday |
| Computability | chore: delete deprecated declarations to March 2026 (#44240) | yesterday |
| Condensed | chore: delete deprecated declarations to March 2026 (#44240) | yesterday |
| Control | chore: rename `Traversable.foldlM/foldrM` to `foldlM/foldrM` (#44641 | yesterday |
| Data | fix(Data/DFinsupp,Order/Preorder/Finsupp): make `coe_strictMono` stat… | 5 hours ago |
| Deprecated | chore: demote public imports that need no downstream imports (#43918) | 2 weeks ago |
| Dynamics | chore: delete deprecated declarations to March 2026 (#44240) | yesterday |
| FieldTheory | feat(RingTheory): generalize relNorm_eq_pow_of_isMaximal to separable… | 7 hours ago |
| Geometry | feat: add `ContMDiffWithinAt.contMDiffWithinAt_tangentMapWithin` and … | 7 hours ago |
| GroupTheory | chore(FreeAbelianGroup): deprecate multiplication (#42249) | yesterday |
| InformationTheory | chore: format imports (#44513) | 2 days ago |
| Lean | chore: delete deprecated declarations to March 2026 (#44240) | yesterday |
| LinearAlgebra | doc(*): use existing citation keys for free-text citations (#44673) | 4 hours ago |
| Logic | feat(Algebra/Notation/Pi): add lemmas on equality conditions and left… | yesterday |
| MeasureTheory | doc(*): use existing citation keys for free-text citations (#44673) | 4 hours ago |
| ModelTheory | chore: remove superfluous focusing dots (#44242) | yesterday |
| NumberTheory | feat(NumberTheory/LSeries): define Hardy's Z function (#42963) | yesterday |
| Order | chore(Order/SuccPred/Relation): use `to_dual` (#44412) | 4 hours ago |
| Probability | chore: remove superfluous focusing dots (#44242) | yesterday |
| RepresentationTheory | chore: remove superfluous focusing dots (#44242) | yesterday |
| RingTheory | doc(*): use existing citation keys for free-text citations (#44673) | 4 hours ago |
| SetTheory | fix(Linter/UnusedTactic): don't mark every tacticSeq as used (#44709) | 17 hours ago |
| Tactic | fix(Linter/UnusedTactic): don't mark every tacticSeq as used (#44709) | 17 hours ago |
| Testing/Plausible | chore: format imports (#44513) | 2 days ago |
| Topology | chore(Topology): fix some spelling errors (#44752) | 7 hours ago |

# Note

# Note

For the rest of the talk, we will mostly be focusing on formal verification rather than formalising mathematics.

# Note

For the rest of the talk, we will mostly be focusing on formal verification rather than formalising mathematics. This is because learning how to create and implement new mathematical structures in Lean is a bit more technical.

# What is the secret ingredient?

# What is the secret ingredient?

Answer: A universe named `Prop`.

# What is the secret ingredient?

Answer: A universe named `Prop`.

Lean has the ability to view every statement as something *like* a set of all proofs of that statement. And these statements live in a universe named `Prop`.

# What is the secret ingredient?

Answer: A universe named `Prop`.

Lean has the ability to view every statement as something *like* a set of all proofs of that statement. And these statements live in a universe named `Prop`.

How does this help?

# Proofs in Lean

# Proofs in Lean

Suppose $P$ is a statement - hence, also the set of all its proofs.

# Proofs in Lean

Suppose $P$ is a statement - hence, also the set of all its proofs. How does this set-theoretic interpretation relate to its truth value?

# Proofs in Lean

Suppose $P$ is a statement - hence, also the set of all its proofs. How does this set-theoretic interpretation relate to its truth value?

- $P$ is *true* precisely when $P \neq \emptyset$.

# Proofs in Lean

Suppose $P$ is a statement - hence, also the set of all its proofs. How does this set-theoretic interpretation relate to its truth value?

- $P$ is *true* precisely when $P \neq \emptyset$.
- $P$ is *false* precisely when $P = \emptyset$.

# Implications in Lean

# Implications in Lean

Fundamentally, what we do in mathematics is assume certain premises (i.e., a statement $P$), and prove theorems (i.e., a statement $Q$).

# Implications in Lean

Fundamentally, what we do in mathematics is assume certain premises (i.e., a statement $P$), and prove theorems (i.e., a statement $Q$).

So, the next question is, is there a way to interpret the statement

$$P \implies Q$$

as a set (using the 'sets' $P$ and $Q$)?

# Implications in Lean

Fundamentally, what we do in mathematics is assume certain premises (i.e., a statement $P$), and prove theorems (i.e., a statement $Q$).

So, the next question is, is there a way to interpret the statement

$$P \implies Q$$

as a set (using the 'sets' $P$ and $Q$)?

Answer: Yes, using functions!

# Implications in Lean

Suppose $P$ and $Q$ are statements. The following two tables show why this makes sense:

# Implications in Lean

Suppose $P$ and $Q$ are statements. The following two tables show why this makes sense:

| $P$ | $Q$ | $P \implies Q$ |
|---|---|---|
| *true* | *true* | *true* |
| *true* | *false* | *false* |
| *false* | *true* | *true* |
| *false* | *false* | *true* |

| $P$ | $Q$ | Functions$(P, Q)$ |
|---|---|---|
| not $\emptyset$ | not $\emptyset$ | not $\emptyset$ |
| not $\emptyset$ | $\emptyset$ | $\emptyset$ |
| $\emptyset$ | not $\emptyset$ | not $\emptyset$ |
| $\emptyset$ | $\emptyset$ | not $\emptyset$ |

# Proving statements in Lean

# Proving statements in Lean

Again, suppose $P$ and $Q$ are statements.

# Proving statements in Lean

Again, suppose $P$ and $Q$ are statements.

Question: How does one prove $P$?

# Proving statements in Lean

Again, suppose $P$ and $Q$ are statements.

Question: How does one prove $P$?
Answer: By producing an element of the set $P$.

# Proving statements in Lean

Again, suppose $P$ and $Q$ are statements.

Question: How does one prove $P$?
Answer: By producing an element of the set $P$.

Question: How does one prove $P \implies Q$?

# Proving statements in Lean

Again, suppose $P$ and $Q$ are statements.

Question: How does one prove $P$?
Answer: By producing an element of the set $P$.

Question: How does one prove $P \implies Q$?
Answer: By producing a function from $P$ to $Q$.

# Moving away from set theory

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs.
Then, why do we want to move away from set theory?

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs. Then, why do we want to move away from set theory?

Most theorem provers use a different foundational language for mathematics called type theory.

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs. Then, why do we want to move away from set theory?

Most theorem provers use a different foundational language for mathematics called type theory.

Theorem provers are mostly built on one of two kinds of type theory:

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs. Then, why do we want to move away from set theory?

Most theorem provers use a different foundational language for mathematics called type theory.

Theorem provers are mostly built on one of two kinds of type theory:

- Simple type theory - Isabelle/HOL.

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs. Then, why do we want to move away from set theory?

Most theorem provers use a different foundational language for mathematics called type theory.

Theorem provers are mostly built on one of two kinds of type theory:

- Simple type theory - Isabelle/HOL.
- Dependent type theory - Lean, Rocq.

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs. Then, why do we want to move away from set theory?

Most theorem provers use a different foundational language for mathematics called type theory.

Theorem provers are mostly built on one of two kinds of type theory:

- Simple type theory - Isabelle/HOL.
- Dependent type theory - Lean, Rocq.

Dependent type theory itself comes in many flavours, e.g. Martin-Löf type theory (MLTT), homotopy type theory (HoTT), calculus of inductive constructions, etc.

# Moving away from set theory

We have seen a set-theoretic interpretation of statements and proofs. Then, why do we want to move away from set theory?

Most theorem provers use a different foundational language for mathematics called type theory.

Theorem provers are mostly built on one of two kinds of type theory:

- Simple type theory - Isabelle/HOL.
- Dependent type theory - **Lean**, Rocq.

Dependent type theory itself comes in many flavours, e.g. Martin-Löf type theory (MLTT), homotopy type theory (HoTT), calculus of inductive constructions, etc.

# Type theory

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

$$\text{elements} \to \text{sets} \to \text{categories}$$

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

$$\text{elements} \to \text{sets} \to \text{categories}$$

There are two things I would like to highlight here:

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

$$\text{elements} \to \text{sets} \to \text{categories}$$

There are two things I would like to highlight here:

1. Every element is a member of *some* set, and every set is an object in *some* category - but this set/category is not well defined.

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

$$\text{elements} \to \text{sets} \to \text{categories}$$

There are two things I would like to highlight here:

1. Every element is a member of *some* set, and every set is an object in *some* category - but this set/category is not well defined.
   For instance: $1 \in \mathbb{N}$, $1 \in \mathbb{Z}$, $1 \in \mathbb{Q}$, etc.

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

$$\text{elements} \to \text{sets} \to \text{categories}$$

There are two things I would like to highlight here:

1. Every element is a member of *some* set, and every set is an object in *some* category - but this set/category is not well defined.
   For instance: $1 \in \mathbb{N}$, $1 \in \mathbb{Z}$, $1 \in \mathbb{Q}$, etc.
2. Every set can be viewed as an element, and every (small) category can be viewed as a set (and hence also an element).

# Type theory

Roughly speaking, in set/category theory/everyday-mathematics, everything lives in three levels of abstraction:

$$\text{elements} \to \text{sets} \to \text{categories}$$

There are two things I would like to highlight here:

1. Every element is a member of *some* set, and every set is an object in *some* category - but this set/category is not well defined.
   For instance: $1 \in \mathbb{N}$, $1 \in \mathbb{Z}$, $1 \in \mathbb{Q}$, etc.
2. Every set can be viewed as an element, and every (small) category can be viewed as a set (and hence also an element).

Type theory preserves the second property, while 'strengthening' the first.

# Type theory

In Lean's type theory, the three levels of abstraction are:

$$\text{terms} \longrightarrow \text{types} \longrightarrow \text{universes}$$

# Type theory

In Lean's type theory, the three levels of abstraction are:

$$\text{terms} \to \text{types} \to \text{universes}$$

Just as in set theory, where every object of interest is an element, here every object of interest is a term.

# Type theory

In Lean's type theory, the three levels of abstraction are:

$$\text{terms} \rightarrow \text{types} \rightarrow \text{universes}$$

Just as in set theory, where every object of interest is an element, here every object of interest is a term.
But, the major difference is that every term is contained in a *unique* type, and every type is contained in a *unique* universe

# Type theory

In Lean's type theory, the three levels of abstraction are:

$$\text{terms} \to \text{types} \to \text{universes}$$

Just as in set theory, where every object of interest is an element, here every object of interest is a term.
But, the major difference is that every term is contained in a *unique* type, and every type is contained in a *unique* universe (this 'strengthens' point 1 from the previous slide).

# Type theory

In Lean's type theory, the three levels of abstraction are:

$$\text{terms} \to \text{types} \to \text{universes}$$

Just as in set theory, where every object of interest is an element, here every object of interest is a term.
But, the major difference is that every term is contained in a *unique* type, and every type is contained in a *unique* universe (this 'strengthens' point 1 from the previous slide).

So, *a priori* $1 \in \mathbb{N}$ and $1 \in \mathbb{Q}$ are not the same 1 in Lean.

# Type theory

But, like sets and categories, types and universes can also be objects of interest.

# Type theory

But, like sets and categories, types and universes can also be objects of interest. Hence, every type is also a term, and every universe is also a type (hence also a term).

# Type theory

But, like sets and categories, types and universes can also be objects of interest. Hence, every type is also a term, and every universe is also a type (hence also a term).

Question: Suppose $X$ is a type, and it is contained in a universe $\mathcal{C}$. Since $X$ is also a term, which unique type is it contained in?

# Type theory

But, like sets and categories, types and universes can also be objects of interest. Hence, every type is also a term, and every universe is also a type (hence also a term).

Question: Suppose $X$ is a type, and it is contained in a universe $\mathcal{C}$. Since $X$ is also a term, which unique type is it contained in?
Answer: $\mathcal{C}$! Because every universe is a type.

# Type theory

But, like sets and categories, types and universes can also be objects of interest. Hence, every type is also a term, and every universe is also a type (hence also a term).

Question: Suppose $X$ is a type, and it is contained in a universe $\mathcal{C}$. Since $X$ is also a term, which unique type is it contained in?
Answer: $\mathcal{C}$! Because every universe is a type. In fact, for any term $x \in X$, we get a unique chain:

$$x \in X \in \mathcal{C} \in \mathcal{C}_1 \in \mathcal{C}_2 \in \mathcal{C}_3 \dots$$

# Terminology and notation

# Terminology and notation

In type theory we do not use the word ‘contained’ when defining terms.

# Terminology and notation

In type theory we do not use the word ‘contained’ when defining terms. This is probably because containment is a statement.

# Terminology and notation

In type theory we do not use the word 'contained' when defining terms.
This is probably because containment is a statement.
Each of $1 \in \mathbb{N}$, $2 \in \mathbb{R}$ and $\pi \in \mathbb{Z}$ is a different meaningful statement, rather than being intrinsic to the definition of 1, 2 and $\pi$ respectively.

# Terminology and notation

In type theory we do not use the word 'contained' when defining terms. This is probably because containment is a statement.
Each of $1 \in \mathbb{N}$, $2 \in \mathbb{R}$ and $\pi \in \mathbb{Z}$ is a different meaningful statement, rather than being intrinsic to the definition of 1, 2 and $\pi$ respectively.

Instead, if $\mathtt{x}$ is a term whose corresponding unique type is $\mathtt{X}$, we say: '$\mathtt{x}$ is a term of type $\mathtt{X}$'.

# Terminology and notation

In type theory we do not use the word 'contained' when defining terms. This is probably because containment is a statement.
Each of $1 \in \mathbb{N}$, $2 \in \mathbb{R}$ and $\pi \in \mathbb{Z}$ is a different meaningful statement, rather than being intrinsic to the definition of 1, 2 and $\pi$ respectively.

Instead, if $\mathtt{x}$ is a term whose corresponding unique type is $\mathtt{X}$, we say: '$\mathtt{x}$ is a term of type $\mathtt{X}$'.

Since we are avoiding the word contained, we are also going to avoid the symbol '$\in$'. Instead, if $\mathtt{x}$ is a term of type $\mathtt{X}$, we shall use the notation

# Terminology and notation

In type theory we do not use the word 'contained' when defining terms. This is probably because containment is a statement.
Each of $1 \in \mathbb{N}$, $2 \in \mathbb{R}$ and $\pi \in \mathbb{Z}$ is a different meaningful statement, rather than being intrinsic to the definition of 1, 2 and $\pi$ respectively.

Instead, if $\mathtt{x}$ is a term whose corresponding unique type is $\mathtt{X}$, we say: '$\mathtt{x}$ is a term of type $\mathtt{X}$'.

Since we are avoiding the word contained, we are also going to avoid the symbol '$\in$'. Instead, if $\mathtt{x}$ is a term of type $\mathtt{X}$, we shall use the notation

$$\mathtt{x} : \mathtt{X}$$

# Terminology and notation

In type theory we do not use the word 'contained' when defining terms. This is probably because containment is a statement.
Each of $1 \in \mathbb{N}$, $2 \in \mathbb{R}$ and $\pi \in \mathbb{Z}$ is a different meaningful statement, rather than being intrinsic to the definition of 1, 2 and $\pi$ respectively.

Instead, if $\mathtt{x}$ is a term whose corresponding unique type is $\mathtt{X}$, we say: '$\mathtt{x}$ is a term of type $\mathtt{X}$'.

Since we are avoiding the word contained, we are also going to avoid the symbol '$\in$'. Instead, if $\mathtt{x}$ is a term of type $\mathtt{X}$, we shall use the notation

$$\mathtt{x} : \mathtt{X}$$

**Intuition:** Even though we do not use the same terminology and notation, it is best to think of terms as elements, types as sets and universes as categories.

# Function types

# Function types

Suppose `X` and `Y` are types.

# Function types

Suppose $\mathtt{X}$ and $\mathtt{Y}$ are types. Then, we have a new type, denoted by $\mathtt{X} \to \mathtt{Y}$, the type of all functions from $\mathtt{X}$ to $\mathtt{Y}$.

# Function types

Suppose $\mathtt{X}$ and $\mathtt{Y}$ are types. Then, we have a new type, denoted by $\mathtt{X} \to \mathtt{Y}$, the type of all functions from $\mathtt{X}$ to $\mathtt{Y}$.

What does this mean?

# Function types

Suppose $\mathtt{X}$ and $\mathtt{Y}$ are types. Then, we have a new type, denoted by $\mathtt{X} \to \mathtt{Y}$, the type of all functions from $\mathtt{X}$ to $\mathtt{Y}$.

What does this mean?
Answer: $\mathtt{X} \to \mathtt{Y}$ is a type such that for any term $\mathtt{f} : \mathtt{X} \to \mathtt{Y}$

# Function types

Suppose $\mathtt{X}$ and $\mathtt{Y}$ are types. Then, we have a new type, denoted by $\mathtt{X} \to \mathtt{Y}$, the type of all functions from $\mathtt{X}$ to $\mathtt{Y}$.

What does this mean?
Answer: $\mathtt{X} \to \mathtt{Y}$ is a type such that for any term $\mathtt{f} : \mathtt{X} \to \mathtt{Y}$, and for any term $\mathtt{x} : \mathtt{X}$,

# Function types

Suppose `X` and `Y` are types. Then, we have a new type, denoted by $\texttt{X} \to \texttt{Y}$, the type of all functions from `X` to `Y`.

What does this mean?
Answer: $\texttt{X} \to \texttt{Y}$ is a type such that for any term $\texttt{f} : \texttt{X} \to \texttt{Y}$, and for any term $\texttt{x} : \texttt{X}$,

$$\texttt{f x} : \texttt{Y}.$$

# Function types

Suppose `X` and `Y` are types. Then, we have a new type, denoted by $\texttt{X} \to \texttt{Y}$, the type of all functions from `X` to `Y`.

What does this mean?
Answer: $\texttt{X} \to \texttt{Y}$ is a type such that for any term $\texttt{f} : \texttt{X} \to \texttt{Y}$, and for any term $\texttt{x} : \texttt{X}$,

$$\texttt{f x} : \texttt{Y}.$$

**Intuition:** $\texttt{X} \to \texttt{Y}$ should be thought of as the set of all functions from `X` to `Y`.

# The universe `Prop`

# The universe `Prop`

`Prop` is a predefined universe in Lean

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`)

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type`

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type` (it also has two other names, `Type 0` and `Sort 1`)

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type` (it also has two other names, `Type 0` and `Sort 1`), and

$$\texttt{Prop} : \texttt{Type}.$$

# The universe `Prop`

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type` (it also has two other names, `Type 0` and `Sort 1`), and

$$\texttt{Prop} : \texttt{Type}.$$

**Intuition:** Just as *Sets* is the category of all sets, `Type` should be viewed as the universe of all types.

# The universe Prop

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q` : `Prop`, the function type `P` $\to$ `Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type` (it also has two other names, `Type 0` and `Sort 1`), and

$$\texttt{Prop} : \texttt{Type}.$$

**Intuition:** Just as *Sets* is the category of all sets, `Type` should be viewed as the universe of all types. In fact, $\mathbb{N}$ : `Type`

# The universe Prop

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q : Prop`, the function type `P → Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type` (it also has two other names, `Type 0` and `Sort 1`), and

$$\texttt{Prop} : \texttt{Type}.$$

**Intuition:** Just as *Sets* is the category of all sets, `Type` should be viewed as the universe of all types. In fact, $\mathbb{N}$ : `Type`, $\mathbb{Q}$ : `Type`

# The universe Prop

`Prop` is a predefined universe in Lean (it has two other names: `Sort` and `Sort 0`), with the property that for any `P`, `Q : Prop`, the function type `P → Q` is a term of type

$$(\texttt{P} \to \texttt{Q}) : \texttt{Prop}.$$

**Intuition:** As discussed before, this should be viewed as an implication (something much more general is true, but we shall ignore that for now).

There is another predefined universe in Lean called `Type` (it also has two other names, `Type 0` and `Sort 1`), and

$$\texttt{Prop} : \texttt{Type}.$$

**Intuition:** Just as *Sets* is the category of all sets, `Type` should be viewed as the universe of all types. In fact, $\mathbb{N}$ : `Type`, $\mathbb{Q}$ : `Type`, $\mathbb{R}$ : `Type`, etc.

# Uniqueness of proofs

# Uniqueness of proofs

Suppose `P : Prop`.

# Uniqueness of proofs

Suppose `P : Prop`.
Then, from our previous discussion we know that we should view `P` as a type whose terms are all the proofs of `P`.

# Uniqueness of proofs

Suppose `P : Prop`.
Then, from our previous discussion we know that we should view `P` as a type whose terms are all the proofs of `P`. So, if our goal is to prove `P`, then we just need to provide a term of type `P`.

# Uniqueness of proofs

Suppose `P : Prop`.
Then, from our previous discussion we know that we should view `P` as a type whose terms are all the proofs of `P`. So, if our goal is to prove `P`, then we just need to provide a term of type `P`.

If

$$\mathtt{h}_1 : \mathtt{P} \text{ and } \mathtt{h}_2 : \mathtt{P}$$

are two proofs

# Uniqueness of proofs

Suppose `P : Prop`.
Then, from our previous discussion we know that we should view `P` as a type whose terms are all the proofs of `P`. So, if our goal is to prove `P`, then we just need to provide a term of type `P`.

If

$$\mathtt{h_1} : \mathtt{P} \text{ and } \mathtt{h_2} : \mathtt{P}$$

are two proofs, then another defining property of `Prop` is that

$$\mathtt{h_1} = \mathtt{h_2}.$$

# Uniqueness of proofs

Suppose `P : Prop`.
Then, from our previous discussion we know that we should view `P` as a type whose terms are all the proofs of `P`. So, if our goal is to prove `P`, then we just need to provide a term of type `P`.

If

$$\mathtt{h_1} : \mathtt{P} \text{ and } \mathtt{h_2} : \mathtt{P}$$

are two proofs, then another defining property of `Prop` is that

$$\mathtt{h_1} = \mathtt{h_2}.$$

**Intuition:** So, `P` should be viewed as a one-element set if true, and empty if false!

# Uniqueness of proofs

Suppose `P : Prop`.
Then, from our previous discussion we know that we should view `P` as a type whose terms are all the proofs of `P`. So, if our goal is to prove `P`, then we just need to provide a term of type `P`.

If

$$\mathtt{h_1} : \mathtt{P} \text{ and } \mathtt{h_2} : \mathtt{P}$$

are two proofs, then another defining property of `Prop` is that

$$\mathtt{h_1} = \mathtt{h_2}.$$

**Intuition:** So, `P` should be viewed as a one-element set if true, and empty if false! In other words, `P` is the set of equivalence classes of proofs of `P`, where any two proofs of `P` are equivalent.

# Examples of propositions

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`.

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

- $1 \leq 2$

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

- $1 \leq 2$
- $2 + 2 = 5$

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

- $1 \leq 2$
- $2 + 2 = 5$
- $\forall\ \mathtt{x} : \mathbb{N},\ 0 \leq \mathtt{x}$

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

- $1 \leq 2$
- $2 + 2 = 5$
- $\forall\ \mathtt{x} : \mathbb{N},\ 0 \leq \mathtt{x}$
- $\forall\ \mathtt{n} : \mathbb{N},\ (\exists\ \mathtt{a}\ \mathtt{b}\ \mathtt{c} : \mathbb{N},\ \mathtt{a}\ \hat{}\ \mathtt{n} + \mathtt{b}\ \hat{}\ \mathtt{n} = \mathtt{c}\ \hat{}\ \mathtt{n}) \to \mathtt{n} \leq 2$

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

- $1 \leq 2$
- $2 + 2 = 5$
- $\forall\ \mathtt{x} : \mathbb{N},\ 0 \leq \mathtt{x}$
- $\forall\ \mathtt{n} : \mathbb{N},\ (\exists\ \mathtt{a}\ \mathtt{b}\ \mathtt{c} : \mathbb{N},\ \mathtt{a}\ \hat{}\ \mathtt{n} + \mathtt{b}\ \hat{}\ \mathtt{n} = \mathtt{c}\ \hat{}\ \mathtt{n}) \to \mathtt{n} \leq 2$
- $\forall\ \mathtt{n} : \mathbb{N},\ \forall\ \mathtt{x}\ \mathtt{y} : \mathbb{R},\ \neg\ (2 \mid \mathtt{n}) \to \mathtt{x}\ \hat{}\ \mathtt{n} = \mathtt{y}\ \hat{}\ \mathtt{n} \to \mathtt{x} = \mathtt{y}$

# Examples of propositions

In the previous slides we used letters to denote terms of type `Prop`. The following are some concrete examples of terms in this universe:

- $1 \leq 2$
- $2 + 2 = 5$
- $\forall\ \mathtt{x} : \mathbb{N},\ 0 \leq \mathtt{x}$
- $\forall\ \mathtt{n} : \mathbb{N},\ (\exists\ \mathtt{a}\ \mathtt{b}\ \mathtt{c} : \mathbb{N},\ \mathtt{a}\ \hat{}\ \mathtt{n} + \mathtt{b}\ \hat{}\ \mathtt{n} = \mathtt{c}\ \hat{}\ \mathtt{n}) \to \mathtt{n} \leq 2$
- $\forall\ \mathtt{n} : \mathbb{N},\ \forall\ \mathtt{x}\ \mathtt{y} : \mathbb{R},\ \neg\ (2 \mid \mathtt{n}) \to \mathtt{x}\ \hat{}\ \mathtt{n} = \mathtt{y}\ \hat{}\ \mathtt{n} \to \mathtt{x} = \mathtt{y}$

# Example 1

# Example 1

Let us start with a very simple one:

# Example 1

Let us start with a very simple one:

Everything that appears above the turnstile ($\vdash$) is a term that is given to you. And whatever is to the right of the turnstile is the statement we need to prove (i.e., provide a term of).

```
P : Prop
h : P
⊢ P
```

term:

# Example 1

Let us start with a very simple one:

Everything that appears above the turnstile ($\vdash$) is a term that is given to you. And whatever is to the right of the turnstile is the statement we need to prove (i.e., provide a term of).

```
P : Prop
h : P
⊢ P
```

term: `h`

# Example 2

# Example 2

**Intuition/Reminder:** As discussed before, we should think of statements as the sets of all proofs, implications as sets of functions, and a proof as an (in fact unique) element of the set of proofs.

# Example 2

**Intuition/Reminder:** As discussed before, we should think of statements as the sets of all proofs, implications as sets of functions, and a proof as an (in fact unique) element of the set of proofs.

```
P Q : Prop
hPQ : P → Q
hP : P
⊢ Q
```

term:

# Example 2

**Intuition/Reminder:** As discussed before, we should think of statements as the sets of all proofs, implications as sets of functions, and a proof as an (in fact unique) element of the set of proofs.

```
P Q : Prop
hPQ : P → Q
hP : P
⊢ Q
```

term: `hPQ hP`

# Example 3

# Example 3

```
P Q R S : Prop
hPQ : P → Q
hQR : Q → R
hRS : R → S
hP : P
⊢ S
```

term:

# Example 3

```
P Q R S : Prop
hPQ : P → Q
hQR : Q → R
hRS : R → S
hP : P
⊢ S
```

term: `hRS (hQR (hPQ hP))`

# Example 3

```
P Q R S : Prop
hPQ : P → Q
hQR : Q → R
hRS : R → S
hP : P
⊢ S
```

term: `hRS (hQR (hPQ hP))`

*Note:* Without the brackets, Lean will interpret it as

$$\texttt{hRS hQR hPQ hP} = \texttt{((hRS hQR) hPQ) hP},$$

which does not make sense. But, one does not need to write: `hPQ (hP)`.

# Example 4

# Example 4

The keyword `fun` can be used to define a term of some function type.

# Example 4

The keyword `fun` can be used to define a term of some function type. For instance, if `X : Type`, then

$$\texttt{fun (x : X)} \mapsto 4$$

is a term of type

$$\texttt{X} \to \mathbb{N}$$

# Example 4

The keyword `fun` can be used to define a term of some function type. For instance, if `X : Type`, then

$$\texttt{fun } (\texttt{x} : \texttt{X}) \mapsto 4$$

is a term of type

$$\texttt{X} \to \mathbb{N}$$

(i.e., the constant function that takes every term of type `X` to $4 : \mathbb{N}$).

# Example 4

The keyword `fun` can be used to define a term of some function type. For instance, if `X : Type`, then

$$\texttt{fun } (\texttt{x} : \texttt{X}) \mapsto 4$$

is a term of type

$$\texttt{X} \to \mathbb{N}$$

(i.e., the constant function that takes every term of type `X` to $4 : \mathbb{N}$).

```
P Q R S : Prop
hPQ : P → Q
hQR : Q → R
hRS : R → S
⊢ P → S
```

term:

# Example 4

The keyword `fun` can be used to define a term of some function type. For instance, if `X : Type`, then

$$\texttt{fun (x : X)} \mapsto 4$$

is a term of type

$$\texttt{X} \to \mathbb{N}$$

(i.e., the constant function that takes every term of type `X` to $4 : \mathbb{N}$).

```
P Q R S : Prop
hPQ : P → Q
hQR : Q → R
hRS : R → S
⊢ P → S
```

term: `fun (hP : P) ↦`

# Example 4

The keyword `fun` can be used to define a term of some function type. For instance, if `X : Type`, then

$$\texttt{fun (x : X)} \mapsto 4$$

is a term of type

$$\texttt{X} \to \mathbb{N}$$

(i.e., the constant function that takes every term of type `X` to $4 : \mathbb{N}$).

```
P Q R S : Prop
hPQ : P → Q
hQR : Q → R
hRS : R → S
⊢ P → S
```

term: `fun (hP : P) ↦ hRS (hQR (hPQ hP))`

# Example 5

# Example 5

Suppose `S T : Prop`, and $\texttt{h}_\texttt{s}$ : `S` and $\texttt{h}_\texttt{t}$ : `T` are proofs of the respective statements.

# Example 5

Suppose $\mathtt{S}\ \mathtt{T} : \mathtt{Prop}$, and $\mathtt{h_s} : \mathtt{S}$ and $\mathtt{h_t} : \mathtt{T}$ are proofs of the respective statements. Then, we have a proof of $\mathtt{S} \wedge \mathtt{T}$, and we denote it by

$$\langle \mathtt{h_s}, \mathtt{h_t} \rangle : \mathtt{S} \wedge \mathtt{T}.$$

# Example 5

Suppose $\mathtt{S}\ \mathtt{T} : \mathtt{Prop}$, and $\mathtt{h_s} : \mathtt{S}$ and $\mathtt{h_t} : \mathtt{T}$ are proofs of the respective statements. Then, we have a proof of $\mathtt{S} \wedge \mathtt{T}$, and we denote it by

$$\langle \mathtt{h_s}, \mathtt{h_t} \rangle : \mathtt{S} \wedge \mathtt{T}.$$

Note: The order is important - because $\langle \mathtt{h_t}, \mathtt{h_s} \rangle : \mathtt{T} \wedge \mathtt{S}$.

# Example 5

Suppose $\mathtt{S}\ \mathtt{T} : \mathtt{Prop}$, and $\mathtt{h_s} : \mathtt{S}$ and $\mathtt{h_t} : \mathtt{T}$ are proofs of the respective statements. Then, we have a proof of $\mathtt{S} \wedge \mathtt{T}$, and we denote it by

$$\langle \mathtt{h_s}, \mathtt{h_t} \rangle : \mathtt{S} \wedge \mathtt{T}.$$

Note: The order is important - because $\langle \mathtt{h_t}, \mathtt{h_s} \rangle : \mathtt{T} \wedge \mathtt{S}$.

$$\begin{aligned}
&\mathtt{n} : \mathbb{N} \\
&\mathtt{hypo} : \mathtt{n} \geq 53 \to \mathtt{n} > 50 \\
&\mathtt{h} : \mathtt{n} > 47 \wedge \mathtt{Nat.Prime\ n} \to \mathtt{n} \geq 53 \\
&\mathtt{h_1} : \mathtt{n} > 47 \\
&\mathtt{h_2} : \mathtt{Nat.Prime\ n} \\
&\vdash \mathtt{n} > 50
\end{aligned}$$

term:

# Example 5

Suppose $\mathtt{S}\ \mathtt{T} : \mathtt{Prop}$, and $\mathtt{h_s} : \mathtt{S}$ and $\mathtt{h_t} : \mathtt{T}$ are proofs of the respective statements. Then, we have a proof of $\mathtt{S} \wedge \mathtt{T}$, and we denote it by

$$\langle \mathtt{h_s}, \mathtt{h_t} \rangle : \mathtt{S} \wedge \mathtt{T}.$$

Note: The order is important - because $\langle \mathtt{h_t}, \mathtt{h_s} \rangle : \mathtt{T} \wedge \mathtt{S}$.

$$\begin{aligned}
&\mathtt{n} : \mathbb{N} \\
&\mathtt{hypo} : \mathtt{n} \geq 53 \to \mathtt{n} > 50 \\
&\mathtt{h} : \mathtt{n} > 47 \wedge \mathtt{Nat.Prime\ n} \to \mathtt{n} \geq 53 \\
&\mathtt{h_1} : \mathtt{n} > 47 \\
&\mathtt{h_2} : \mathtt{Nat.Prime\ n} \\
&\vdash \mathtt{n} > 50
\end{aligned}$$

term: $\mathtt{hypo\ (h\ \langle h_1,\ h_2 \rangle)}$

# Interesting example?

# Interesting example?

Seeing these, you might guess that this can get complicated very easily

# Interesting example?

Seeing these, you might guess that this can get complicated very easily - and you are right!

# Interesting example?

Seeing these, you might guess that this can get complicated very easily - and you are right! For instance, consider (the `inst` below says that `G` has a multiplicative group structure):

```
G : Type
inst : Group G
⊢ 1⁻¹ = 1
```

term:

# Interesting example?

Seeing these, you might guess that this can get complicated very easily - and you are right! For instance, consider (the `inst` below says that `G` has a multiplicative group structure):

```
G : Type
inst : Group G
⊢ 1⁻¹ = 1
```

term:

```
have this := fun {a b} h ↦ have this' := Eq.mpr (id
(congrArg (fun _a ↦ a⁻¹ * a * b = a⁻¹ * _a) (Eq.symm h)))
(Eq.mpr (id (congrArg (fun _a ↦ _a = a⁻¹ * (a * b))
(mul_assoc a⁻¹ a b))) (Eq.refl (a⁻¹ * (a * b)))); Eq.mpr
(id (congrArg (fun _a ↦ a⁻¹ = _a) (Eq.mp (congrArg (fun _a
↦ b = _a) (mul_one a⁻¹)) (Eq.mp (congrArg (fun _a ↦ _a =
a⁻¹ * 1) (one_mul b)) (Eq.mp (congrArg (fun _a ↦ _a * b =
a⁻¹ * (1 :  G)) (inv_mul_cancel a)) this')))))) (Eq.refl
a⁻¹); this (Eq.mpr (id (congrArg (fun _a ↦ _a = 1) (one_mul
1))) (Eq.refl 1))
```

Is there a better way to prove statements?

Is there a better way to prove statements?
Yes, using *tactics*!

# Tactics

# Tactics

Tactics are commands in Lean which help simplify the problem.

# Tactics

Tactics are commands in Lean which help simplify the problem. For instance, suppose you have the following problem (from Example 2):

```
P Q : Prop
hPQ : P → Q
hP : P
⊢ Q
```

# Tactics

Tactics are commands in Lean which help simplify the problem. For instance, suppose you have the following problem (from Example 2):

```
P Q : Prop
hPQ : P → Q
hP : P
⊢ Q
```

Using a tactic called `apply` (which we will see later), we can tell Lean:

*We know that* `P` $\implies$ `Q` *is true. Hence, it is enough to prove* `P` *to prove* `Q`*. So, can you simplify the goal to* `P`*?*

# Tactics

Tactics are commands in Lean which help simplify the problem. For instance, suppose you have the following problem (from Example 2):

$$\begin{aligned} &\texttt{P Q} : \texttt{Prop} \\ &\texttt{hPQ} : \texttt{P} \to \texttt{Q} \\ &\texttt{hP} : \texttt{P} \\ &\vdash \texttt{Q} \end{aligned}$$

Using a tactic called `apply` (which we will see later), we can tell Lean:

*We know that* $\texttt{P} \implies \texttt{Q}$ *is true. Hence, it is enough to prove* `P` *to prove* `Q`*. So, can you simplify the goal to* `P`*?*

And, that's an easier goal because we already have a proof of the statement `P`.

# Tactics

Tactics are commands in Lean which help simplify the problem. For instance, suppose you have the following problem (from Example 2):

$$\begin{aligned} &\texttt{P Q} : \texttt{Prop} \\ &\texttt{hPQ} : \texttt{P} \to \texttt{Q} \\ &\texttt{hP} : \texttt{P} \\ &\vdash \texttt{Q} \end{aligned}$$

Using a tactic called `apply` (which we will see later), we can tell Lean:

*We know that* $\texttt{P} \implies \texttt{Q}$ *is true. Hence, it is enough to prove* `P` *to prove* `Q`*. So, can you simplify the goal to* `P`*?*

And, that's an easier goal because we already have a proof of the statement `P`.

Great, now that we have some intuition for what's going on, we can start playing around!

# Interesting links

# Interesting links

Learning material:

1. If you want to get comfortable playing around with tactics, I recommend Kevin Buzzard's lecture notes (link).

# Interesting links

Learning material:

1. If you want to get comfortable playing around with tactics, I recommend Kevin Buzzard's lecture notes (link).
2. If you want to learn how to build mathematical objects from the ground up in Lean, I recommend Avigad and Massot's book, *Mathematics in Lean* (link, pdf).

# Interesting links

Learning material:

1. If you want to get comfortable playing around with tactics, I recommend Kevin Buzzard's lecture notes (link).
2. If you want to learn how to build mathematical objects from the ground up in Lean, I recommend Avigad and Massot's book, *Mathematics in Lean* (link, pdf).
3. If you want to understand the fundamentals of type theory more, and how it works in Lean, I recommend the book by Avigad, de Moura, Kong, Ullrich and others, *Theorem Proving in Lean 4* (link).

# Interesting links

Learning material:

1. If you want to get comfortable playing around with tactics, I recommend Kevin Buzzard's lecture notes (link).
2. If you want to learn how to build mathematical objects from the ground up in Lean, I recommend Avigad and Massot's book, *Mathematics in Lean* (link, pdf).
3. If you want to understand the fundamentals of type theory more, and how it works in Lean, I recommend the book by Avigad, de Moura, Kong, Ullrich and others, *Theorem Proving in Lean 4* (link).

Probably the most important links for anything Lean-related are:

# Interesting links

Learning material:

1. If you want to get comfortable playing around with tactics, I recommend Kevin Buzzard's lecture notes (link).
2. If you want to learn how to build mathematical objects from the ground up in Lean, I recommend Avigad and Massot's book, *Mathematics in Lean* (link, pdf).
3. If you want to understand the fundamentals of type theory more, and how it works in Lean, I recommend the book by Avigad, de Moura, Kong, Ullrich and others, *Theorem Proving in Lean 4* (link).

Probably the most important links for anything Lean-related are:

- The Lean community website: leanprover-community.github.io

# Interesting links

Learning material:

1. If you want to get comfortable playing around with tactics, I recommend Kevin Buzzard's lecture notes (link).
2. If you want to learn how to build mathematical objects from the ground up in Lean, I recommend Avigad and Massot's book, *Mathematics in Lean* (link, pdf).
3. If you want to understand the fundamentals of type theory more, and how it works in Lean, I recommend the book by Avigad, de Moura, Kong, Ullrich and others, *Theorem Proving in Lean 4* (link).

Probably the most important links for anything Lean-related are:

- The Lean community website: leanprover-community.github.io
- The Zulip chat group: leanprover.zulipchat.com

# Project references

Here are links to all the projects mentioned during the talk:

# Project references

Here are links to all the projects mentioned during the talk:

- Lean perfectoid spaces:
  leanprover-community.github.io/lean-perfectoid-spaces/

# Project references

Here are links to all the projects mentioned during the talk:

- Lean perfectoid spaces:
  leanprover-community.github.io/lean-perfectoid-spaces/
- Liquid Tensor Experiment (the challenge):
  xenaproject.wordpress.com/2020/12/05/liquid-tensor-experiment/

# Project references

Here are links to all the projects mentioned during the talk:

- Lean perfectoid spaces:
  leanprover-community.github.io/lean-perfectoid-spaces/
- Liquid Tensor Experiment (the challenge):
  xenaproject.wordpress.com/2020/12/05/liquid-tensor-experiment/
- Completion of the Liquid Tensor Experiment:
  leanprover-community.github.io/blog/posts/lte-final/

# Project references

Here are links to all the projects mentioned during the talk:

- Lean perfectoid spaces:
  leanprover-community.github.io/lean-perfectoid-spaces/
- Liquid Tensor Experiment (the challenge):
  xenaproject.wordpress.com/2020/12/05/liquid-tensor-experiment/
- Completion of the Liquid Tensor Experiment:
  leanprover-community.github.io/blog/posts/lte-final/
- Fermat's Last Theorem (led by Kevin Buzzard):
  github.com/ImperialCollegeLondon/FLT

# Project references

Here are links to all the projects mentioned during the talk:

- Fermat's Last Theorem (Anthropic's announcement): anthropic.com/research/formalizing-fermats-last-theorem

# Project references

Here are links to all the projects mentioned during the talk:

- Fermat's Last Theorem (Anthropic's announcement): anthropic.com/research/formalizing-fermats-last-theorem
- Navier-Stokes (OpenAI's announcement): openai.com/index/navier-stokes-solution/

# Project references

Here are links to all the projects mentioned during the talk:

- Fermat's Last Theorem (Anthropic's announcement): anthropic.com/research/formalizing-fermats-last-theorem
- Navier-Stokes (OpenAI's announcement): openai.com/index/navier-stokes-solution/
- Poincaré conjecture (announcement): frenzymath.com/news/poincare-formalization/

# Project references

Here are links to all the projects mentioned during the talk:

- Fermat's Last Theorem (Anthropic's announcement): anthropic.com/research/formalizing-fermats-last-theorem
- Navier-Stokes (OpenAI's announcement): openai.com/index/navier-stokes-solution/
- Poincaré conjecture (announcement): frenzymath.com/news/poincare-formalization/
- Mathlib4: github.com/leanprover-community/mathlib4

*Thank you!*